



Gymnázium J. K. Tyla

AI-asistovaný softwarový vývoj – Agentické inženýrství a harness DarkFactory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D
Vedoucí práce: Michal Dočekal

Rozsah práce: 5300 slov / 35221 znaků

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Odborná práce zkoumá využití agentní umělé inteligence při vývoji softwaru se zaměřením na architekturu agentního harnessu. Kapitoly 2–4 sledují přechod od jazykového modelu přes Harness k AI-asistovanému vývoji a agentickému inženýrství. Kapitoly 5–6 oddělují kanonickou dokumentaci DarkFactory od vyhodnocení dostupných důkazů. Evaluace používá zdrojový kód, automatické testy a CI výsledky a výslovně rozlišuje prokázané mechanismy od neprokázaného plného produkčního průchodu.

Abstract

This thesis examines the use of agentic artificial intelligence in software development, focusing on the architecture of an agent harness. Chapters 2–4 progress from the language model through the harness to AI-assisted development and agentic engineering. Chapters 5–6 separate the canonical DarkFactory documentation from evaluation of the available evidence. The evaluation uses source code, automated tests, and CI results and explicitly distinguishes demonstrated mechanisms from a full production lifecycle that was not demonstrated.

Keywords (Klíčová slova)

.agents/, .claude/, Agentní sezení (Session), Agentní smyčka (Agent Loop), AGENTS.md, CLAUDE.md, Cílené smyčky (Goal Loops), Degradace kontextu (Context Rot), Dovednosti (Skills), Graf pracovního postupu (Workflow Graph), Guardrail, Harness, Hooks, Inferenční engine (Inference Engine), Integrovaný test (Integration Test), Izolované prostředí (Sandbox), Kompakce kontextu (Context Compaction), Kontextové inženýrství (Context Engineering), Kontextové okno (Context Window), MCP, Mezipaměť klíčů a hodnot (KV Cache), Nástroje (Tools), Plánování (Planning), Poskytovatel modelu (Model Provider), Prompt Injection, Promptové inženýrství (Prompt Engineering), Prostředí agenta (Agent Environment), Průběžná integrace (CI), Pull Request, Předání řízení (Handoff), Přepis (Transcript), RAG, Revize (Review), Slop, Spouštění kódu (Code Execution), Správa verzí (Version Control), Stav (State), Subagent, Swarm, Systémový prompt (System Prompt), Teplota (Temperature), Token, Transformer, Vektorová reprezentace (Embedding), Velký jazykový model (LLM), Vibe Coding, Vkládání kontextu (Context Injection), Vyvolávání nástrojů (Tool Calling), Vývoj řízený specifikací (Spec-Driven Development), Větev (Branch), Člověk ve smyčce (HITL)

Obsah

Prohlášení	2
Anotace	3
Abstract	4
Keywords (Klíčová slova)	5
1 Úvod	8
1.1 Motivace a vymezení problému	8
1.2 Východisko a argument práce	8
1.3 Cíle	8
1.3.1 Hlavní cíl	8
1.3.2 Dílčí cíle	9
1.4 Výzkumné otázky	9
1.5 Metodika	9
2 Jazykový model	11
2.1 Architektura a reprezentace	11
2.2 Inference	12
3 Harness	14
3.1 Smyčka a stav	14
3.2 Prostředí a nástroje	15
3.3 Rozšíření	16
4 AI-asistovaný vývoj a agentické inženýrství	19
4.1 Zadání a způsob práce	19
4.2 Řízení změny	20
4.3 Kvalita a ověřování	21
4.4 Instrukce a kontext	21
4.5 Řízení agentního chování	23
4.6 Orchestrace agentů	24
5 DarkFactory	26
6 Vyhodnocení	27
6.1 Ověření mechanismů	27
6.2 Ověření systému	27
6.3 Ověření na repozitářích	27
6.4 Výzkumné otázky	28
6.5 Diskuse a omezení	28
7 Závěr	29

Seznam obrázků a tabulek	30
Seznam příloh	31
A Encyklopedie a rejstřík pojmů	32
Seznam zdrojů	36

1 Úvod

Práce se zabývá použitím agentní AI při vývoji softwaru a technickými podmínkami, které umožňují delegovat delší úlohy bez ztráty kontroly nad stavem, účinky a ověřením výsledku.

Předmětem práce není trénování jazykových modelů, ale systémové vrstvy potřebné pro jejich praktické použití jako součásti agentního vývojového procesu. Východisko a argument, podle kterého je tato otázka dále rozpracována, stanovuje část 1.2.

1.1 Motivace a vymezení problému

Současné agentní systémy dokážou nad softwarovým projektem provádět více kroků, pracovat se soubory a nástroji a ověřovat vlastní změny. Hlavním problémem této práce proto není samotné generování kódu, ale technická vrstva, která převádí schopnosti jazykového modelu do řízeného, obnovitelného a ověřitelného vývojového procesu.⁽¹⁾ Současné agentní architektury propojují model s nástroji, pracovním prostředím a dlouhodobějším stavem, takže mohou provádět více navazujících kroků místo izolované textové odpovědi.^(1,2)

Rozšíření generativní AI zároveň vytváří praktický důvod tyto systémy studovat: Microsoft AI Economy Institute pro první čtvrtletí roku 2026 odhadl používání generativní AI na 17,8 % světové populace ve věku 15–64 let.⁽³⁾ Tento údaj v práci slouží pouze jako kontext rozšíření technologie, nikoli jako metrika kvality nebo autonomie agentních systémů. DarkFactory slouží jako konkrétní artefakt, na kterém práce zkoumá oddělení modelu, trvalého stavu, prostředí, nástrojů a kontrolních mechanismů.

1.2 Východisko a argument práce

Rostoucí schopnosti jazykových modelů mění vývoj softwaru od dílčí asistence k delegování stále delších a samostatnějších pracovních úloh. Argument práce postupuje v jedné návaznosti: jak AI mění způsob zadávání, provádění a ověřování softwarové práce; co samotný model skutečně je a dokáže; co kolem něj přidává Harness; jak Agentické inženýrství tyto mechanismy záměrně skládá a omezuje; jak je realizuje DarkFactory; a co o výsledku skutečně ukazují dostupné důkazy.

1.3 Cíle

Tato část stanovuje hlavní a dílčí cíle. Cíle vymezují, jaký artefakt má být navržen a které jeho vlastnosti mají být technicky ověřeny. Výzkumné otázky jsou odděleny do následující samostatné části.

1.3.1 Hlavní cíl

Navrhnout a implementovat architekturu Harnessu pro dlouhotrvající vývoj softwaru, která odděluje jazykový model od trvalého stavu, prostředí a deterministických kontrolních mechanismů, a technicky ověřit vlastnosti této architektury na systému DarkFactory.

DarkFactory je implementační artefakt této architektury. Splnění cíle se posuzuje podle dohledatelné implementace navržených mechanismů a reprodukovatelných testovacích nebo provozních důkazů. Úplný živý životní cyklus změny je samostatná úroveň ověření a nesmí být zaměněn za samotnou existenci architektury nebo úspěšné komponentové testy.

1.3.2 Dílčí cíle

- Vymezit teoretické mechanismy AI-asistovaného vývoje, jazykového modelu a inference, Harnessu a Agentického inženýrství potřebné pro dlouhotrvající agentní vývoj softwaru.
- Navrhnout a implementovat DarkFactory s explicitním stavem běhu, odděleným prostředím, rozšiřitelným capability rozhraním, GitHubem jako řídicí vrstvou a oddělenými hranicemi lidské a strojové identity.
- Implementovat mechanismy řízeného životního cyklu změny: plánování, deterministické ověření, smyčku revize a opravy, finální kontrolu souladu, integraci a obnovu přerušného běhu.
- Ověřit implementované mechanismy automatickými testy a CI nad konkrétním commitem.
- Ověřit přenositelnost vybraných částí řešení na konkrétních cílových repozitářích a samostatně vyhodnotit, zda existuje důkaz úplného živého životního cyklu změny.
- Vztáhnout zjištěné výsledky a jejich omezení přímo k výzkumným otázkám.

Dílčí cíle oddělují teoretické vymezení, konstrukci artefaktu a jednotlivé úrovně jeho ověření. Nesplněná nebo neprokázaná úroveň evaluace proto nemusí být nahrazena silnějším tvrzením z jiné vrstvy důkazů.

1.4 Výzkumné otázky

- O1: Které mechanismy agentního harnessu a vývojového životního cyklu umožňují agentovi samostatně provádět softwarovou změnu, zatímco rozhodnutí s vyšším dopadem zůstávají explicitně řízena člověkem?
- O2: Jak může agentní harness omezit nebo obnovit neproduktivní či přerušovaný běh bez ztráty již ověřeného stavu a bez opakování přijatých deterministických účinků?
- O3: Jak lze oddělit trvalý stav dlouhotrvající úlohy od omezeného kontextového okna modelu tak, aby bylo možné práci po přerušení bezpečně obnovit a pokračovat v ní?

O1 sleduje řízenou autonomii, O2 odolnost provádění a O3 kontinuitu stavu přes hranice jednotlivých modelových kontextů a běhů. Otázky jsou záměrně formulovány tak, aby na ně bylo možné odpovědět konkrétními architektonickými prvky a reprodukovatelnými důkazy z implementace.

1.5 Metodika

Práce používá konstrukční přístup odpovídající design science: vymezuje problém a cíle řešení, navrhuje a implementuje artefakt DarkFactory a následně jej vyhodnocuje pomocí reprodukovatelných technických důkazů. ^(4,5)

Teoretická část vychází z odborných článků, standardů, protokolových specifikací a primární dokumentace současných agentních systémů. Jednotlivé mechanismy jsou rozděleny do samostatných konceptů, aby bylo možné oddělit vlastnosti **Velký jazykový model (LLM)**, mechanismy Harnessu a postupy Agentického inženýrství.

Předmětem práce není trénování neuronových sítí, optimalizace vah ani matematický rozbor učení modelu. **Velký jazykový model (LLM)** je chápán jako hotová inferenční komponenta a je popsán pouze v rozsahu potřebném pro vysvětlení architektury okolního systému.

Praktická část používá zdrojový kód, typované kontrakty, testy, pracovní postupy a generované artefakty DarkFactory jako primární důkaz skutečné implementace. Evaluace rozlišuje architektonický důkaz, automatizované funkční testy, CI nad konkrétním commitem a provozní ověření na konkrétních cílových repozitářích. Tvrzení o úplném produkčním životním cyklu změny je přijato pouze tehdy, pokud existuje reprodukovatelný živý průchod od schválení plánu po sloučení změny a rekonsiliaci; aktuální absence tohoto důkazu je uvedena jako omezení, nikoli nahrazena architektonickým předpokladem.

Práce neprovádí statistický benchmark obecné výkonnosti agentních systémů. Odpovědi na výzkumné otázky jsou omezeny na tvrzení podporovaná konkrétními reprodukovatelnými důkazy.

2 Jazykový model

Tato kapitola odděluje vlastnosti jazykového modelu, jeho běhové inference a limity, které z tohoto způsobu zpracování vyplývají.

Modelová vrstva končí vytvořením výstupu nad aktuálním kontextem. Trvalý stav úlohy, účinky v externím prostředí, nástroje a dlouhodobé řízení patří až do Harnessu.

2.1 Architektura a reprezentace

Velký jazykový model (LLM)

Velký neuronový jazykový model trénovaný na rozsáhlých textových datech pro predikci a generování posloupností tokenů. ⁽⁶⁾

Současné LLM typicky používají architekturu **Transformer**, která při autoregresivním generování odhaduje další token z předchozí sekvence. ⁽⁷⁾ *Praktický význam:* V agentickém vývoji je model zdrojem návrhů a rozhodnutí, ale sám nezajišťuje trvalý stav ani provedení účinků mimo inferenci.

Transformer

Architektura neuronové sítě založená na mechanismu pozornosti, který modeluje vztahy mezi prvky sekvence. ⁽⁷⁾

Při autoregresivním generování dekodér z dosavadní sekvence vytváří reprezentaci použitou k odhadu následujícího tokenu. ⁽⁷⁾ *Praktický význam:* Architektura Transformer umožňuje zpracovat kontext, nad nímž agent vytváří další kroky; délka a kvalita tohoto kontextu proto přímo ovlivňují dostupné rozhodování.

Tokenizér

Komponenta, která převádí vstup na posloupnost tokenů a jejich identifikátorů a provádí odpovídající zpětné dekódování. ⁽⁸⁾

Pravidla tokenizace určují slovník a segmentaci vstupu; subword metody, například Byte Pair Encoding, umožňují skládat text z jednotek menších než celé slovo. ⁽⁸⁾ *Praktický význam:* Tokenizace určuje, jak se vstup a výstup počítají do kontextového rozpočtu, takže ovlivňuje délku instrukcí, historie i nástrojových výsledků.

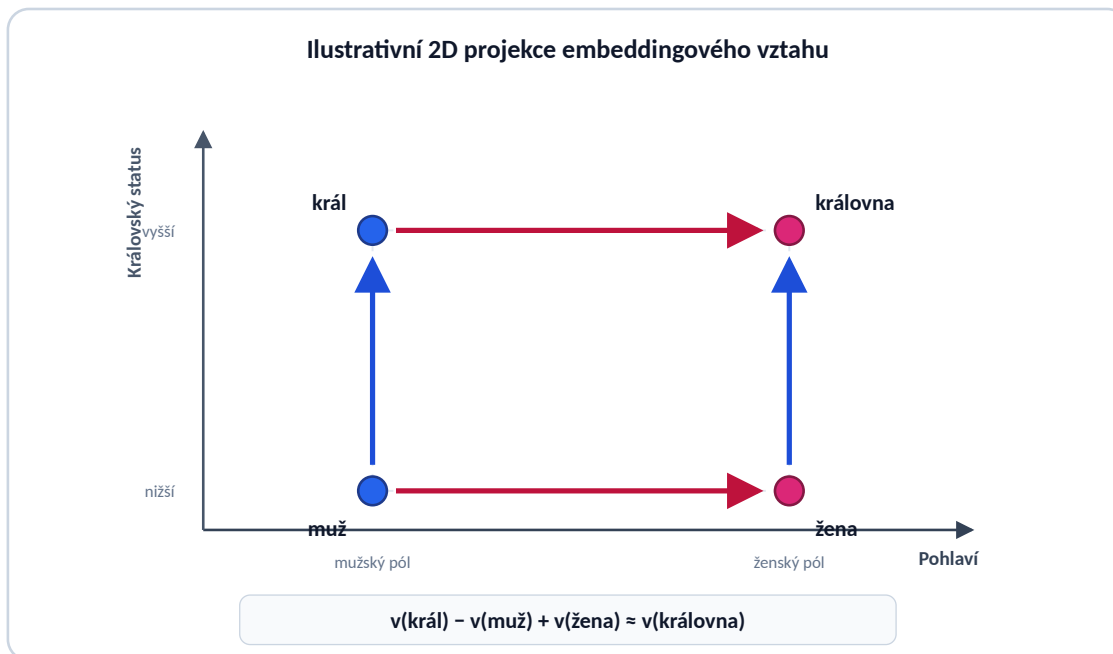
Token

Diskrétní jednotka vstupní nebo výstupní sekvence reprezentovaná identifikátorem ve slovníku tokenizéru. ⁽⁸⁾

Token nemusí odpovídat celému slovu; subword tokenizace může pracovat s částmi slov nebo jinými textovými fragmenty. ⁽⁸⁾ Identifikátor tokenu je před zpracováním neuronovou sítí převeden na vektorovou reprezentaci. *Praktický význam:* Token je jednotka kontextového rozpočtu a generování; agentní systém proto musí řídit, kolik tokenů spotřebují instrukce, historie a výsledky nástrojů.

Vektorová reprezentace (Embedding)

Vícerozměrná vektorová reprezentace diskrétních prvků, v níž geometrické vztahy mohou zachycovat užitečné vztahy mezi reprezentovanými objekty. ⁽⁹⁾
V jazykovém modelu embedding převádí identifikátory tokenů na spojité vektory zpracovatelné neuronovou sítí; vektorové reprezentace lze také porovnávat podle podobnosti. ⁽⁹⁾



Obrázek 1: Ilustrativní 2D projekce vztahu $\text{král} - \text{muž} + \text{žena} \approx \text{královna}$; osy Pohlaví a Královský status slouží pouze k názornému vysvětlení a nepředstavují doslovné naučené dimenze embeddingového prostoru.
Praktický význam: Vektorové reprezentace umožňují vyhledávat podle sémantické podobnosti, což je praktický základ pro výběr relevantního kontextu například v retrieval pipeline.

2.2 Inference

Poskytovatel modelu (Model Provider)

Poskytovatel modelu je rozhraní nebo služba, přes kterou agentní runtime vybírá a volá konkrétní model. OpenAI Agents SDK například odděluje modelové implementace od providerů a podporuje OpenAI i jiné providery. ⁽¹⁰⁾
Provider zprostředkovává přístup k modelu a jeho konkrétnímu API; dostupné funkce a nastavení se proto mohou mezi providery lišit. ⁽¹⁰⁾ *Praktický význam:* Volba poskytovatele určuje dostupné modely, rozhraní, limity a provozní podmínky, se kterými musí Harness při volání modelu pracovat.

Inferenční engine (Inference Engine)

Běhová vrstva, která načte jazykový model a provádí jeho inferenci nad vstupními tokeny. ⁽¹¹⁾

Inferenční engine zajišťuje praktické provedení modelu, například plánování inferenčních požadavků, správu výpočetních prostředků a práci s mezipamětí během generování. ⁽¹¹⁾ *Praktický význam:* Inferenční engine převádí požadavek na skutečné provedení modelu a ovlivňuje dostupné parametry, latenci a způsob obsluhy dlouhých agentních běhů.

Teplota (Temperature)

Teplota je parametr vzorkování, který u rozhraní, jež jej podporují, ovlivňuje náhodnost výběru dalších tokenů. ⁽¹²⁾

OpenAI Responses API dokumentuje rozsah 0 až 2; vyšší hodnoty zvyšují náhodnost a nižší hodnoty vedou k soustředěnějšímu a determinističtějšímu výstupu. ⁽¹²⁾ *Praktický význam:* Teplota umožňuje řídit míru náhodnosti vzorkování tam, kde ji dané rozhraní podporuje; agentní systém ji proto může volit podle požadované stability nebo variability výstupu.

Kontextové okno (Context Window)

Maximální rozsah tokenové sekvence dostupný modelu v jednom inferenčním běhu. ⁽¹³⁾ Do aktivního kontextu mohou vstupovat instrukce, uživatelský vstup i další data předaná systému pro daný inferenční běh. Samotná nominální délka okna nezaručuje, že model všechny vložené informace využije stejně spolehlivě. ⁽¹³⁾ *Praktický význam:* Kontextové okno omezuje množství instrukcí, historie a pozorování dostupných modelu v jednom kroku, takže agent musí kontext vybírat a průběžně spravovat.

Mezipaměť klíčů a hodnot (KV Cache)

Mezipaměť dříve vypočtených klíčů a hodnot v pozornostních vrstvách transformeru používaná při autoregresivním generování. ⁽¹⁴⁾ Při generování dalšího tokenu lze uložené klíče a hodnoty předchozí sekvence znovu použít místo jejich úplného přepočítání; paměťové nároky cache přitom rostou s délkou aktivní sekvence. ^(14,15) *Praktický význam:* KV cache umožňuje znovu využívat již zpracovaný prefix během generování, což může snižovat režii opakovaných inferenčních kroků v dlouhém sezení.

Degradace kontextu (Context Rot)

V této práci označuje Context Rot pokles spolehlivosti, s nímž model využívá relevantní informace v dlouhém nebo zahlceném kontextu. ⁽¹³⁾ Experimenty s dlouhým kontextem ukazují, že výkon může záviset na poloze relevantní informace a klesat, když je umístěna uvnitř dlouhého vstupu. ⁽¹³⁾ *Praktický význam:* Degradace kontextu znamená, že pouhé zvětšování historie nemusí zachovat kvalitu rozhodování; agentní systém proto potřebuje selekci, sumarizaci nebo kompakci. Modelová vrstva poskytuje inferenční výstup, nikoli kontinuitu dlouhotrvající úlohy ani provedení účinků v externím prostředí. Přejít od izolovaného modelového volání k systému schopnému dlouhodobě jednat proto vyžaduje další vrstvu: Harness.

3 Harness

Harness je běhová vrstva kolem modelové inference, která drží stav, opakuje agentní smyčku a propojuje model s nástroji a prostředím. ⁽²⁾

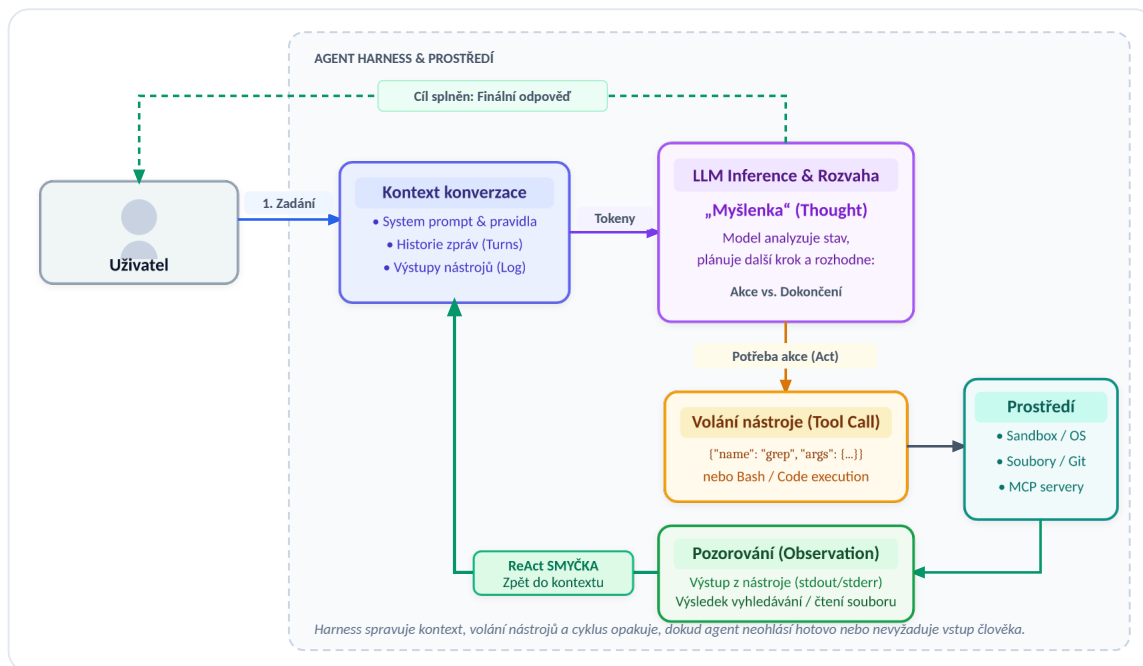
Jeho odpovědností je kontinuita běhu a provedení účinků mimo model. Strategie, podle které se tyto schopnosti skládají do cíleného chování, patří do agentického inženýrství. *Praktický význam:* Harness umožňuje převést jednotlivé modelové inference na dlouhotrvající agentní běh, který může udržovat stav, používat nástroje a pracovat se skutečným prostředím.

3.1 Smyčka a stav

Agentní smyčka (Agent Loop)

Iterativní cyklus, v němž model vyhodnotí stav, zvolí akci, harness ji provede a výsledek vrátí do další iterace. ⁽¹⁶⁾

ReAct formalizuje střídání rozhodnutí, akce a pozorování výsledku; nové pozorování se stává vstupem dalšího kroku. ⁽¹⁶⁾



Obrázek 2: Smyčka ReAct: model zvolí akci, harness ji provede a výsledek vrátí modelu.

Praktický význam: Agentní smyčka umožňuje opakovaně převádět pozorování na další akci, takže agent může postupovat po více krocích místo jednorázové odpovědi.

Agentní sezení (Session)

Persistovaná jednotka, která vymezuje jeden souvislý agentní běh a umožňuje jeho pozdější pokračování. ⁽¹⁷⁾

Session je vlastníkem identity a hranice pokračujícího běhu; jeho historický průběh vlastní **Přepis (Transcript)** a aktuální pracovní skutečnosti **Stav (State)**.^(2,17) *Praktický význam:* Sezení dává více krokům společnou kontinuitu, takže lze navázat na předchozí položky, nástroje a stav bez zakládání zcela nového běhu.

Přepis (Transcript)

Uspořádaný historický záznam událostí vzniklých během **Agentní sezení (Session)**, například zpráv, akcí a výsledků nástrojů.⁽²⁾ Transcript odpovídá na otázku, co se během běhu stalo. Je historickým záznamem, nikoli reprezentací právě platného pracovního stavu.⁽²⁾ *Praktický význam:* Přepis poskytuje auditovatelnou historii interakcí a nástrojových událostí, z níž lze rekonstruovat průběh běhu a hledat příčiny chyb.

Stav (State)

Persistovaná reprezentace aktuálně platných pracovních skutečností a řídicích údajů běhu.⁽²⁾ State odpovídá na otázku, co je pro další krok právě platné; na rozdíl od **Přepis (Transcript)** nemusí zachovávat úplnou historii předchozích událostí.⁽²⁾ *Praktický význam:* Stav umožňuje pokračovat podle aktuálně platných skutečností a řídicích údajů, aniž by bylo nutné spoléhat na to, že vše zůstane v textové historii modelu.

3.2 Prostředí a nástroje

Prostředí agenta (Agent Environment)

Vnější prostředí, které agent prostřednictvím harnessu pozoruje a mění, například pracovní soubory, procesy, síťové služby a další systémové prostředky.⁽²⁾ Změna souboru nebo spuštění procesu mění stav prostředí mimo modelový kontext. Přístup k těmto účinkům zprostředkovávají **Nástroje (Tools)** a jejich bezpečnostní hranice.⁽²⁾ *Praktický význam:* Prostředí dává agentovi konkrétní pracovní prostor, v němž může číst soubory, spouštět příkazy a pozorovat skutečné výsledky.

Nástroje (Tools)

Rozhraní, kterým agent vyvolává operace mimo samotnou textovou inferenci, například čtení dat, volání API nebo změnu stavu systému.⁽¹⁾ Harness zprostředkuje požadavek na nástroj, provede operaci v prostředí a vrátí její výsledek modelu jako další pozorování.^(1,18) *Praktický význam:* Nástroje umožňují agentovi spouštět příkazy, číst soubory, volat API, spouštět testy a pracovat se skutečnými výsledky místo jejich predikování.

Vývolávání nástrojů (Tool Calling)

Mechanismus, kterým model místo běžné textové odpovědi vybere konkrétní **Nástroje (Tools)** a vytvoří strukturované argumenty pro jeho vyvolání. ⁽¹⁾ Schéma rozhraní omezuje tvar argumentů a umožňuje jejich programovou validaci; JSON Schema je jedním z používaných formátů takového kontraktu. ⁽¹⁹⁾ *Praktický význam:* Tool Calling propojuje rozhodnutí modelu s deterministicky provedenou funkcí nebo službou a vrací skutečný výsledek zpět do dalšího kroku.

Spouštění kódu (Code Execution)

Nástrojová schopnost umožňující vykonat program nebo příkaz a vrátit jeho skutečný výstup modelu. ⁽²⁰⁾ Při vývoji softwaru zpřístupňuje agentovi například testy, buildy, formátovače a diagnostické příkazy místo odhadování jejich výsledku. ⁽²⁰⁾ *Praktický význam:* Spouštění kódu umožňuje agentovi ověřovat hypotézy příkazy, testy a programy namísto pouhého predikování jejich výsledku.

Izolované prostředí (Sandbox)

Oddělené běhové prostředí, ve kterém agent může spouštět kód nebo měnit pracovní soubory bez přímého přístupu ke všem prostředkům hostitelského systému. ⁽²⁾ Sandbox vytváří bezpečnostní hranici kolem nedůvěryhodných účinků; konkrétní realizace může používat například kontejner nebo microVM. ^(2, 21) *Praktický význam:* Sandbox omezuje dopad chybného nebo nežádoucího kroku tím, že vymezuje dostupný souborový systém, síť a další schopnosti prostředí.

3.3 Rozšíření

Dovednosti (Skills)

Znovupoužitelný balíček instrukcí a volitelných zdrojů, který se načítá pro úlohy odpovídající jeho účelu. ⁽²²⁾ Agent Skill je definován souborem `SKILL.md` s YAML frontmatterem a instrukcemi v Markdownu; může odkazovat na doplňující skripty, reference nebo další zdroje načítané podle potřeby. ⁽²²⁾ *Praktický význam:* Skills umožňují opakovaně balit doménový postup a podpůrné prostředky tak, aby je agent mohl použít konzistentně bez opakovaného zadávání celé instrukce.

Plugin

Distribuovatelné rozšíření, které do hostitelského agentního prostředí přidává další chování nebo integrace. ⁽²³⁾

Konkrétní platforma může plugin použít jako obal pro různé druhy rozšíření, například skills, hooks, agenty nebo konfiguraci externích integrací. Plugin zde proto označuje způsob balení a distribuce, nikoli nadřazenou kategorii všech mechanismů Harnessu. ⁽²³⁾ *Praktický význam:* Plugin sdružuje rozšiřující schopnosti do distribuovatelné jednotky, takže lze přidat nový pracovní postup nebo integraci bez změny samotného modelu.

Skript

Spustitelné soubory nebo posloupnosti příkazů používané k deterministickému provedení opakovatelné operace. ⁽²³⁾

Agentní rozšíření mohou skripty používat pro transformace, validace nebo jiné kroky, které je výhodnější provést programově než novým modelovým rozhodnutím. ^(22, 23) *Praktický význam:* Skript je vhodný pro deterministické kroky, které mají být provedeny přesně a opakovatelně, například validaci nebo transformaci souborů.

Hooks

Konfigurované reakce spouštěné při určených událostech životního cyklu agentního prostředí. ⁽²⁴⁾

Hook může před nebo po vybrané události spustit deterministickou logiku, například validaci, příkaz nebo jinou automatizaci. ⁽²⁴⁾ *Praktický význam:* Hooks umožňují spustit deterministickou kontrolu nebo reakci v definovaném bodě životního cyklu a vynutit chování nezávisle na tom, zda jej model sám navrhne.

MCP

Otevřený protokol pro standardizované propojení AI aplikací s externími nástroji a datovými zdroji. ⁽²⁵⁾

MCP odděluje klientskou AI aplikaci od serverů poskytujících nástroje a další schopnosti, takže integrace lze implementovat mimo vlastní jádro harnessu. ^(25, 26) *Praktický význam:* MCP standardizuje připojení externích nástrojů, zdrojů a promptů k agentnímu hostiteli, takže integrace nemusí být navržena zvlášť pro každý modelový klient.

.agents/

`.agents/` je repozitářový nebo uživatelský jmenný prostor Codexu pro znovupoužitelná agentní rozšíření, zejména Skills. ⁽²⁷⁾

Repozitářové Skills se ukládají do `.agents/skills` a uživatelské do `~/agents/skills`; AGENTS.md zůstává samostatnou vrstvou projektových instrukcí a není podadresářem `.agents/`. ⁽²⁷⁾ *Praktický význam:* Adresář `.agents/` umožňuje držet repozitářové dovednosti a jejich podpůrné prostředky blízko kódu, který je používá, a verzovat je společně s projektem.

.claude/

`.claude/` je projektový nebo uživatelský jmenný prostor Claude Code pro instrukce, pravidla, nastavení a rozšíření. ^(28,29) Anthropic dokumentuje `.claude/CLAUDE.md`, modulární `.claude/rules/` a projektové `.claude/settings.json`; hooks a Skills jsou samostatné rozšiřující mechanismy s vlastními pravidly. ^(24,28-30) *Praktický význam:* Adresář `.claude/` umožňuje verzovat projektová pravidla, nastavení a rozšíření Claude Code společně s repozitářem. Harness doplňuje modelovou inferenci o kontinuitu běhu a rozhraní pro pozorování a změnu externího prostředí. Tím vzniká stavový agentní systém schopný jednat; výběr kontextu, řízení chování a rozdělování práce jsou navazujícími návrhovými rozhodnutími agentického inženýrství.

4 AI-asistovaný vývoj a agentické inženýrství

AI ve vývoji softwaru posouvá část práce od přímého psaní k zadávání, delegování a kontrole změn. Agentické inženýrství označuje návrh způsobu, jakým se schopnosti Harnessu skládají, omezují a koordinují tak, aby agent cíleně plnil delší úlohu.⁽³¹⁾ Kapitola spojuje způsob formulace práce, řízení změny a nezávislé ověření kvality s návrhem instrukcí, kontextu, řízení chování a orchestrace agentů. *Praktický význam:* Agentické inženýrství převádí obecné schopnosti modelu a Harnessu do opakovatelného vývojového procesu s explicitním zadáním, kontrolami, řízeným kontextem a koordinací práce.

4.1 Zadání a způsob práce

Vibe Coding

Způsob tvorby softwaru, při kterém člověk iteruje pomocí pokynů v přirozeném jazyce bez průběžné kontroly vygenerovaného kódu.⁽³²⁾

Termín zavedl Andrej Karpathy v roce 2025. V této práci slouží jako kontrast k postupům, které před delegováním práce explicitně formulují specifikaci, plán a podmínky ověření.^(32, 33)

Tweet Andreje Karpathyho z 2. února 2025, ve kterém popsal původní význam Vibe Coding.



Andrej Karpathy ✓
@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

3:17 PM · Feb 2, 2025 · 6.8M Views

Obrázek 3: Původní tweet Andreje Karpathyho o Vibe Coding.^(32, 34)

Praktický význam: Vibe Coding může zrychlit průzkumné prototypování, ale bez explicitních kontrol zvyšuje význam následné revize, testů a sledovatelnosti změn.

Vývoj řízený specifikací (Spec-Driven Development)

Přístup k AI-asistovanému vývoji, ve kterém explicitní specifikace řídí plánování, implementaci a ověřování změny.⁽³⁵⁾

Specifikace odděluje požadované chování a omezení od konkrétní implementace a slouží jako společný referenční bod pro člověka i agenta.⁽³⁵⁾ *Praktický význam:* Specifikace dává agentovi explicitní cíl a akceptační podmínky, podle nichž lze plánovat kroky a ověřovat výsledek.

Plánování (Planning)

Převod požadavku na explicitní kroky, závislosti a podmínky ověření před prováděním změn.⁽³⁶⁾

Plán rozděluje práci na kontrolovatelné části, určuje jejich pořadí a stanovuje podmínky, podle kterých lze posoudit dokončení.⁽³⁶⁾ *Praktický význam:* Plánování převádí zadání na pořadí kroků a podmínky ověření, podle nichž může agent postupovat a průběžně kontrolovat dokončení.

Revize (Review)

Revize je samostatná kontrola změny nebo výstupu proti explicitním požadavkům a kvalitativním kritériím před jeho přijetím. GitHub Pull Request review například umožňuje změny komentovat, schválit nebo vrátit s požadavkem na úpravy před sloučením.⁽³⁷⁾ *Praktický význam:* Revize poskytuje samostatný kontrolní krok proti požadavkům a kvalitativním kritériím, takže nalezené odchylky lze vrátit k opravě před přijetím změny.

4.2 Řízení změny

Správa verzí (Version Control)

Systém pro zaznamenávání a porovnávání historie změn souborů v čase.⁽³⁸⁾ Uložená historie umožňuje identifikovat původ změny, vracet se k předchozím stavům a slučovat samostatně vzniklé změny.⁽³⁸⁾ *Praktický význam:* Správa verzí umožňuje agentním změnám zůstat dohledatelné, porovnatelné a vratné místo přepisování pracovního stavu bez historie.

Větev (Branch)

Oddělená linie vývoje v systému správy verzí, která ukazuje na vlastní posloupnost commitů.⁽³⁸⁾ Větev umožňuje izolovat souběžnou změnu od cílové větve a později ji sloučit po kontrole nebo ověření.^(38,39) *Praktický význam:* Větev izoluje rozpracovanou agentní změnu od hlavní historie a vytváří bezpečný prostor pro testování a revizi před integrací.

Pull Request

Návrh na sloučení změn z jedné větve do jiné, kolem kterého GitHub soustřeďuje revizi, diskusi a automatické kontroly. ⁽⁴⁰⁾

Pull request zpřístupňuje diff navržené změny a její stav před integrací do cílové větve.

⁽⁴⁰⁾ *Praktický význam:* Pull Request vytváří explicitní integrační a revizní hranici, kde lze porovnat změny, spustit kontroly a zaznamenat rozhodnutí před sloučením.

4.3 Kvalita a ověřování

Slop

Neformální označení pro nekvalitní digitální obsah, zejména obsah vytvořený umělou inteligencí. ⁽⁴¹⁾

V této práci označuje zejména AI-generovaný software, jehož objem nebo zdánlivá úplnost převyšují jeho ověřenou funkčnost a udržitelnost. *Praktický význam:* Riziko nekvalitního generovaného výstupu znamená, že agentní práce potřebuje objektivní kontroly a revizi namísto přijetí výsledku jen proto, že je syntakticky úplný.

Průběžná integrace (CI)

Vývojová praxe, při níž se změny průběžně integrují a automaticky ověřují sestavením, testy a dalšími kontrolami. ⁽⁴²⁾

CI převádí část podmínek kvality do opakovatelných strojově vyhodnotitelných kontrol spouštěných nad změnami. ⁽⁴²⁾ *Praktický význam:* CI převádí opakovatelné kontroly změny do automatického signálu, který může agent i člověk použít při rozhodování o dalším kroku.

Integrační test (Integration Test)

Ověření spolupráce více komponent nebo vrstev systému přes jejich rozhraní. ⁽³⁶⁾ Integrační test zachycuje chyby vznikající ve vzájemném propojení částí systému, které izolované testování jednotlivých komponent nemusí odhalit. ⁽³⁶⁾ *Praktický význam:* Integrační test ověřuje spolupráci více částí systému, a proto zachytí chyby, které izolovaná kontrola jednotlivého modulu nebo generovaného souboru neodhalí.

4.4 Instrukce a kontext

Promptové inženýrství (Prompt Engineering)

Systematický návrh instrukcí, příkladů a jejich struktury s cílem ovlivnit chování jazykového modelu. ⁽⁴³⁾

Prompt ovlivňuje pravděpodobnostní chování modelu, ale sám nevynucuje technickou bezpečnostní nebo autorizační hranici; pravidla, která musí systém garantovat, patří do **Guardrail** nebo jiné běhové kontroly. *Praktický význam:* Promptové inženýrství umožňuje zpřesnit instrukce pro jeden modelový krok; u agentů je však třeba jeho účinek kombinovat se stavem, nástroji a řízením kontextu.

Systémový prompt (System Prompt)

Systémová instrukční vrstva, která vymezuje roli, pravidla a výchozí způsob chování modelu nebo agenta. ⁽⁴³⁾

Systémový prompt poskytuje stabilní instrukční kontext, ale sám o sobě není technickou izolační ani autorizační hranicí. ⁽⁴³⁾ *Praktický význam:* Systémový prompt stanovuje základní instrukce a hranice chování, které Harness přikládá ke každému relevantnímu modelovému kroku.

AGENTS.md

AGENTS.md je mechanismus projektových instrukcí Codexu, který dodává agentovi trvalý repozitářový kontext před zahájením práce. ⁽⁴⁴⁾ Codex hledá instrukce od kořene repozitáře směrem k aktuálnímu pracovnímu adresáři, skládá je v tomto pořadí a bližší instrukce tak dostávají vyšší prioritu; na úrovni adresáře může AGENTS.override.md nahradit AGENTS.md. ⁽⁴⁴⁾ *Praktický význam:* AGENTS.md umožňuje udržovat repozitářové instrukce přímo u kódu a automaticky je přidávat do kontextu Codexu podle pracovního umístění.

CLAUDE.md

CLAUDE.md je soubor trvalých instrukcí a kontextu, který Claude Code načítá do sezení. ⁽²⁸⁾

Projektové instrukce mohou být v `./CLAUDE.md` nebo `./.claude/CLAUDE.md`, uživatelské v `~/.claude/CLAUDE.md`; soubory nad pracovním adresářem se načítají při spuštění a soubory v podadresářích se mohou načíst až při práci v nich. Jde o kontextové instrukce, nikoli o vynucenou bezpečnostní hranici. ⁽²⁸⁾ *Praktický význam:* CLAUDE.md umožňuje udržovat trvalé projektové nebo uživatelské instrukce, které Claude Code načítá do kontextu sezení.

Kontextové inženýrství (Context Engineering)

Systematický výběr a správa informací, které jsou modelu zpřístupněny v aktivním kontextu během inference. ⁽³¹⁾

Aktivní kontext může obsahovat instrukce, popisy dostupných nástrojů, externí data a vybranou historii interakce. Jeho obsah je nutné kurátorovat vzhledem k omezené kapacitě a nerovnoměrnému využití dlouhého kontextu. ^(13, 31) *Praktický význam:* Kontextové inženýrství rozhoduje, které informace mají být v daném kroku modelu skutečně dostupné, což je zásadní pro dlouhé agentní běhy s omezeným rozpočtem.

Vkládání kontextu (Context Injection)

V této práci označuje Context Injection cílené vložení relevantních informací do aktivního kontextu až v okamžiku, kdy jsou potřebné pro aktuální krok. ⁽³¹⁾

Just-in-time přístup umožňuje mimo modelový kontext uchovávat odkazy nebo trvalý stav a potřebná data načíst nástrojem až během běhu. ⁽³¹⁾ *Praktický význam:* Vkládání kontextu umožňuje Harnessu doplnit modelu aktuální data, instrukce nebo výsledky nástrojů právě v okamžiku, kdy jsou relevantní.

Kompakce kontextu (Context Compaction)

Zmenšení aktivního kontextu nahrazením části historie kratší reprezentací, typicky shrnutím nebo výběrem důležitých informací. ⁽³¹⁾ Kompakce uvolňuje kapacitu pro další běh, ale příliš agresivní komprese může odstranit detaily, které se později ukážou jako důležité. ^(31,45) *Praktický význam:* Kompakce kontextu umožňuje dlouhotrvajícímu agentovi pokračovat přes bezprostřední kontextový rozpočet, za cenu možného ztracení detailů.

RAG

Architektura, ve které systém před generováním vyhledá relevantní informace z externího zdroje a poskytne je modelu jako další kontext. ⁽⁴⁶⁾ RAG odděluje znalost uloženou v externím korpusu od parametrů modelu a umožňuje vybírat podklady podle aktuálního dotazu. ⁽⁴⁶⁾ *Praktický význam:* RAG umožňuje před generováním dohledat relevantní externí informace a přidat je do kontextu, takže agent nemusí spoléhat pouze na parametry modelu.

Prompt Injection

Manipulace chování jazykového modelu pomocí instrukcí vložených do vstupu nebo do externího obsahu, který systém následně zpracuje jako kontext. ⁽⁴⁷⁾ Přímá prompt injection přichází v uživatelském vstupu; nepřímá injection je vložena do externích dat, například webové stránky, e-mailu, dokumentu, repozitáře nebo zdroje RAG. ⁽⁴⁸⁾ Typickým příkladem je životopis obsahující skrytou instrukci, která se pokusí ovlivnit následné hodnocení kandidáta modelem, přestože dokument měl sloužit pouze jako data. ⁽⁴⁷⁾ Důsledky proto závisejí také na oprávněních a nástrojích, které má agent k dispozici. ⁽⁴⁹⁾ *Praktický význam:* Prompt Injection vyžaduje oddělovat důvěryhodné instrukce od nedůvěryhodného obsahu a omezovat následné nástrojové akce, protože text z prostředí může ovlivnit rozhodování modelu.

4.5 Řízení agentního chování

Cílené smyčky (Goal Loops)

V této práci označují řídicí smyčky, které opakují jednání podle explicitního cíle, pozorovaného výsledku a podmínky dalšího pokračování nebo ukončení.

Na rozdíl od samotného běhového Agent Loopu zahrnuje Goal Loop také zpětnou vazbu k dosažení cíle, ověření výsledku a rozhodnutí, zda pokračovat, změnit postup nebo běh ukončit. ^(1,16) *Praktický význam:* Cílená smyčka umožňuje opakovat plánování a ověřování až do splnění explicitní podmínky místo ukončení po prvním přijatelně vypadajícím výstupu.

Guardrail

V této práci označuje Guardrail programově vynucenou kontrolu, která může před pokračováním běhu validovat nebo zablokovat vstup, výstup či použití nástroje. ⁽⁵⁰⁾ Agentní frameworky mohou guardrails implementovat různými způsoby; zde je důležitá deterministicky vyhodnotitelná hranice oddělená od samotného modelového rozhodnutí. Kontrola může běh zastavit nebo odmítnout konkrétní akci před provedením jejího účinku. ⁽⁵⁰⁾ *Praktický význam:* Guardrails umožňují deterministickými kontrolami vynucovat omezení, která nemají záviset pouze na tom, zda je model dodrží.

Člověk ve smyčce (HITL)

Uspořádání automatizovaného procesu, ve kterém člověk v určených bodech poskytuje zpětnou vazbu, schválení nebo rozhodnutí. ⁽⁵¹⁾ HITL ponechává vybraná rozhodnutí člověku místo úplné automatizace. V agentním workflow může být lidský zásah explicitní přechod nebo schvalovací bod, po kterém automatizované provádění pokračuje. ⁽⁵¹⁾ *Praktický význam:* HITL umožňuje vyžádat lidské rozhodnutí před citlivou nebo nevratnou akcí a tím vložit explicitní schvalovací hranici do jinak automatického běhu.

4.6 Orchestrace agentů

Subagent

Specializovaná agentní instance, které jiný agent nebo orchestrátor deleguje vymezenou dílčí úlohu. ⁽⁵²⁾ Subagent umožňuje oddělit roli, instrukce a pracovní kontext dílčí úlohy od koordinujícího běhu a následně vrátit výsledek zpět nadřazené orchestrace. ^(52, 53) *Praktický význam:* Subagent umožňuje oddělit dílčí úlohu do samostatného kontextu a paralelizovat nebo specializovat práci bez zahlcení hlavního agentního vlákna.

Orchestrátor

Koordinační role nebo komponenta, která rozhoduje, kterému specializovanému agentovi předat dílčí práci a jak jeho výsledek začlenit do pokračujícího běhu. ⁽⁵²⁾ Centralizovaný orchestrátor zůstává vlastníkem hlavního workflow a může specialisty volat jako omezené pracovní jednotky, případně jejich práci kombinovat nebo spouštět paralelně. ^(1, 52) *Praktický význam:* Orchestrátor rozděluje práci, spouští dílčí vykonavatele a skládá jejich výsledky, čímž umožňuje koordinovat více agentních větví jako jeden proces.

Předání řízení (Handoff)

Vzor koordinace, při kterém aktivní agent předá další řízení specializovanému agentovi.

(52)

Handoff se liší od centralizované orchestrace tím, že specialista není pouze zavolán jako dílčí pracovní jednotka a vrácen orchestrátoru, ale přebírá aktivní pokračování interakce nebo úlohy. ⁽⁵²⁾ *Praktický význam:* Handoff předává odpovědnost i potřebný kontext jinému agentovi nebo roli, takže další krok nemusí pokračovat ve stejném agentním vlákne.

Graf pracovního postupu (Workflow Graph)

Explicitní grafová reprezentace vícefázového workflow, v níž uzly představují dílčí práci a hrany určují přechody nebo závislosti mezi kroky. ⁽⁵³⁾

Workflow Graph může vyjádřit sekvenční, podmíněné i paralelní větvení a může koordinovat více agentů. Pokud hrany vyjadřují pouze acyklické závislosti, může mít podobu DAG; workflow s návraty nebo opakováním však obecně DAG být nemusí. ^(1, 53) *Praktický význam:* Workflow Graph explicitně zachycuje návaznosti, větvení, paralelismus a smyčky, takže složitější agentní proces lze řídit jako strukturovaný tok práce.

Swarm

Swarm označuje dynamicky koordinované paralelní provádění úlohy více subagenty pod orchestrujícím agentem nebo systémem; nejde o univerzální formální standard. Kimi K2.5 Agent Swarm je konkrétní realizace, v níž orchestrátor dynamicky vytváří a koordinuje paralelní subagenty bez předem definovaných rolí nebo ručně napsaného workflow. ⁽⁵⁴⁾ *Praktický význam:* Swarm umožňuje dynamicky rozdělit rozsáhlou úlohu mezi více paralelních subagentů, pokud zadání přirozeně obsahuje nezávislé části práce. Delegování práce AI nezmenšuje potřebu softwarově-inženýrských kontrol; přesouvá jejich význam k explicitnímu zadání, sledovatelným změnám, řízenému kontextu a strojově ověřitelné zpětné vazbě. Tyto principy vytvářejí základ pro konkrétní realizaci v systému DarkFactory.

5 DarkFactory

6 Vyhodnocení

Vyhodnocení odděluje důkazy o jednotlivých mechanismech od důkazů o jejich spolupráci na úrovni systému a od přenosu na konkrétní cílové repozitáře.

Metodika těchto důkazů je stanovena v části 1.5; zde se už neopakuje. Vyhodnocení používá reprodukovatelný evidence snapshot DarkFactory na commitu `e9c10221b40589512d262a0edb95f709b923150c` a odpovídající CI run `35616745304`. Snapshot slouží pouze jako pevný referenční bod pro výsledky. ^(55,56)

6.1 Ověření mechanismů

Referenční CI run dokončil všech 15 jobů úspěšně a hlavní Bun testovací sada vykázala 670 úspěšných testů ve 102 souborech bez selhání. ⁽⁵⁶⁾ Samostatné testy ověřují mimo jiné persistenci Run State, detekci neaktuálního Planningu, zachování historie při provider failoveru, odvozování výsledku z pracovního stromu, kontrolu scope a recovery provenance. ⁽⁵⁵⁾ Tato evidence podporuje konkrétní mechanismy; sama o sobě neprokazuje úspěšnost celého produkčního životního cyklu.

6.2 Ověření systému

Integrační testy dokazují spolupráci hlavních runtime mechanismů, ale evidence uzavřená 21. září 2026 neobsahuje jeden reprodukovatelný živý průchod celým produkčním životním cyklem změny. Testy pokrývají workflow přechody, review/fix iterace, persistovaný stav, GitHub události, required-check gate, nástrojové účinky, failover a recovery. ⁽⁵⁵⁾ Plný df-only průchod od schválení Planningu po merge a následnou rekonsiliaci zůstal zároveň otevřenou acceptance položkou Requestu 359, a proto není v této práci považován za prokázaný. ⁽⁵⁷⁾

6.3 Ověření na repozitářích

Fleet-level ověření rozlišuje aktivní cílové repozitáře od historických nebo archivovaných položek a hodnotí pouze konkrétní commity s dohledatelnými workflow výsledky. Repozitář `omnis` byl ověřen na commitu `a53660a1c0c6619f94768e5d520405052fb03df6`. Jeho pipeline run `34708160162` dokončil úspěšně mimo jiné web, paper a docs jobs; samostatné deploy-docs a release joby na stejném commitu byly rovněž úspěšné. ^(58,59) Repozitář `ChessWithQuests` byl ověřen na commitu `50a50797f29c2a636d973981993191a65df3d131`. Pipeline run `34708180783` dokončil úspěšně paper, web a docs jobs a na stejném commitu uspěly také verify-docs, deploy-docs a release kontroly. ^(60,61) Původně uváděné repozitáře `template-OdbornaPrace` a `OdbornaPrace-mono` jsou v době evaluace archivované a nejsou proto považovány za aktivní fleet cíle. ^(62,63)

Aktivním repozitářem práce je `DarkFactory-Paper`, který nahrazuje starší označení `OdbornaPrace-paper`. Snapshot `5bc04974f9aed0f55295389154124a09059ff35e` prošel úspěšně CI, Deploy Documentation i Release workflow. ^(64–67) Aktivní fleet evidence tedy v této fázi pokrývá DarkFactory, omnis, ChessWithQuests a DarkFactory-Paper. Dvě původní fleet položky jsou archivované; úplné 361 fleet acceptance proto nelze z těchto výsledků považovat za uzavřené.

6.4 Výzkumné otázky

Odpovědi jsou omezeny na vlastnosti doložené implementací, automatickými testy, CI a dostupnými provozními artefakty.

01. Řízenou autonomii podporuje oddělení schváleného plánu, implementace, deterministického ověření, review/fix smyčky, Final Alignment a merge autorizace. Jejich dílčí kontrakty jsou implementované a testované, ale evidence neobsahuje jeden úplný živý produkční průchod. ^(55–57)

02. Přerušný nebo neproduktivní běh lze omezovat pomocí budgetů, klasifikace chyb a failoveru a obnovovat nad persistovaným stavem s kontrolou provenance. Testy tyto mechanismy podporují, nikoli však živou crash/resume idempotenci nad celým produkčním workflow. ^(55, 57)

03. Trvalý pracovní stav je oddělen od context window: Run State, Session a Transcript mohou existovat mimo aktuální inferenční vstup a aktivní kontext z nich vybírá pouze informace potřebné pro další krok. Tato architektura podporuje pokračování přes hranice modelových běhů, ale nedokazuje nulovou informační ztrátu pro libovolně dlouhou úlohu. ⁽⁵⁵⁾

6.5 Diskuse a omezení

Dostupná evidence podporuje realizaci a testované vlastnosti jednotlivých mechanismů, nikoli obecné tvrzení o úplné autonomii nebo výkonnostní převaze DarkFactory. Práce neprovádí statistický benchmark úspěšnosti, ceny, latence ani četnosti zacyklení na reprezentativním souboru úloh. Evidence také neobsahuje jeden živý plný Request lifecycle ani úplnou acceptance původně plánované fleety. ⁽⁵⁷⁾ Silnou stránkou evaluace je naopak přímá vazba konkrétních tvrzení na implementační a testovací důkazy, například pozorovaný stav pracovního stromu nebo recovery provenance. ⁽⁵⁵⁾ Evaluace prokazuje implementaci a automatické ověření klíčových mechanismů DarkFactory a současně vymezuje chybějící systémové důkazy. Hlavní cíl je proto hodnocen na úrovni navržené architektury a reprodukovatelně testovaných mechanismů; neprovedený plný živý lifecycle zůstává explicitním omezením závěrů. ⁽⁵⁷⁾

7 Závěr

Práce ukazuje, že delegování softwarové práce agentní AI vyžaduje více než samotnou schopnost jazykového modelu generovat kód: rozhodující jsou mechanismy, které mimo model udržují stav, zprostředkovávají účinky a poskytují ověřitelnou zpětnou vazbu. DarkFactory demonstruje konkrétní realizaci tohoto přístupu a dostupná evaluace podporuje funkčnost klíčových mechanismů i jejich vybraných integrací. Výzkumné otázky proto lze uzavřít na úrovni řízené autonomie, obnovitelnosti běhu a oddělení trvalého stavu od omezeného modelového kontextu.

Rozsah závěru je omezen provedenými důkazy: práce neprokazuje obecnou převahu DarkFactory ani jeden úplný živý průchod celým produkčním životním cyklem změny. Tato hranice je součástí výsledku evaluace, nikoli nahrazena silnějším tvrzením. ⁽⁵⁷⁾

Seznam obrázků a tabulek

Obrázek 1	Ilustrativní 2D projekce vztahu král – muž + žena $\approx$ královna; osy Pohlaví a Královský status slouží pouze k názornému vysvětlení a nepředstavují doslovné naučené dimenze embeddingového prostoru.	12
Obrázek 2	Smyčka ReAct: model zvolí akci, harness ji provede a výsledek vrátí modelu. .	14
Obrázek 3	Původní tweet Andreje Karpathyho o Vibe Coding. ^(32, 34)	19

Seznam příloh

A Encyklopedie a rejstřík pojmů	32
---------------------------------------	----

A Encyklopedie a rejstřík pojmů

Abecední přehled klíčových pojmů použitých v práci. Názvy a definice jsou odvozeny přímo z kanonických sémantických položek.

.agents/

`.agents/` je repozitářový nebo uživatelský jmenný prostor Codexu pro znovupoužitelná agentní rozšíření, zejména Skills. ⁽²⁷⁾

.claude/

`.claude/` je projektový nebo uživatelský jmenný prostor Claude Code pro instrukce, pravidla, nastavení a rozšíření. ^(28, 29)

Agentní sezení (Session)

Persistovaná jednotka, která vymezuje jeden souvislý agentní běh a umožňuje jeho pozdější pokračování. ⁽¹⁷⁾

Agentní smyčka (Agent Loop)

Iterativní cyklus, v němž model vyhodnotí stav, zvolí akci, harness ji provede a výsledek vrátí do další iterace. ⁽¹⁶⁾

AGENTS.md

AGENTS.md je mechanismus projektových instrukcí Codexu, který dodává agentovi trvalý repozitářový kontext před zahájením práce. ⁽⁴⁴⁾

CLAUDE.md

CLAUDE.md je soubor trvalých instrukcí a kontextu, který Claude Code načítá do sezení. ⁽²⁸⁾

Cílené smyčky (Goal Loops)

V této práci označují řídicí smyčky, které opakují jednání podle explicitního cíle, pozorování výsledku a podmínky dalšího pokračování nebo ukončení.

Degradace kontextu (Context Rot)

V této práci označuje Context Rot pokles spolehlivosti, s nímž model využívá relevantní informace v dlouhém nebo zahlceném kontextu. ⁽¹³⁾

Dovednosti (Skills)

Znovupoužitelný balíček instrukcí a volitelných zdrojů, který se načítá pro úlohy odpovídající jeho účelu. ⁽²²⁾

Graf pracovního postupu (Workflow Graph)

Explicitní grafová reprezentace vícefázového workflow, v níž uzly představují dílčí práci a hrany určují přechody nebo závislosti mezi kroky. ⁽⁵³⁾

Guardrail

V této práci označuje Guardrail programově vynucenou kontrolu, která může před pokračováním běhu validovat nebo zablokovat vstup, výstup či použití nástroje. ⁽⁵⁰⁾

Harness

Harness je běhová vrstva kolem modelové inference, která drží stav, opakuje agentní smyčku a propojuje model s nástroji a prostředím. ⁽²⁾

Hooks

Konfigurované reakce spouštěné při určených událostech životního cyklu agentního prostředí. ⁽²⁴⁾

Inferenční engine (Inference Engine)

Běhová vrstva, která načte jazykový model a provádí jeho inferenci nad vstupními tokeny. ⁽¹¹⁾

Integrační test (Integration Test)

Ověření spolupráce více komponent nebo vrstev systému přes jejich rozhraní. ⁽³⁶⁾

Izolované prostředí (Sandbox)

Oddělené běhové prostředí, ve kterém agent může spouštět kód nebo měnit pracovní soubory bez přímého přístupu ke všem prostředkům hostitelského systému. ⁽²⁾

Kompakce kontextu (Context Compaction)

Zmenšení aktivního kontextu nahrazením části historie kratší reprezentací, typicky shrnutím nebo výběrem důležitých informací. ⁽³¹⁾

Kontextové inženýrství (Context Engineering)

Systematický výběr a správa informací, které jsou modelu zpřístupněny v aktivním kontextu během inference. ⁽³¹⁾

Kontextové okno (Context Window)

Maximální rozsah tokenové sekvence dostupný modelu v jednom inferenčním běhu. ⁽¹³⁾

MCP

Otevřený protokol pro standardizované propojení AI aplikací s externími nástroji a datovými zdroji. ⁽²⁵⁾

Mezipaměť klíčů a hodnot (KV Cache)

Mezipaměť dříve vypočtených klíčů a hodnot v pozornostních vrstvách transformeru používaná při autoregresivním generování. ⁽¹⁴⁾

Nástroje (Tools)

Rozhraní, kterým agent vyvolává operace mimo samotnou textovou inferenci, například čtení dat, volání API nebo změnu stavu systému. ⁽¹⁾

Plánování (Planning)

Převod požadavku na explicitní kroky, závislosti a podmínky ověření před prováděním změn. ⁽³⁶⁾

Poskytovatel modelu (Model Provider)

Poskytovatel modelu je rozhraní nebo služba, přes kterou agentní runtime vybírá a volá konkrétní model. OpenAI Agents SDK například odděluje modelové implementace od providerů a podporuje OpenAI i jiné providery. ⁽¹⁰⁾

Prompt Injection

Manipulace chování jazykového modelu pomocí instrukcí vložených do vstupu nebo do externího obsahu, který systém následně zpracuje jako kontext. ⁽⁴⁷⁾

Promptové inženýrství (Prompt Engineering)

Systematický návrh instrukcí, příkladů a jejich struktury s cílem ovlivnit chování jazykového modelu. ⁽⁴³⁾

Prostředí agenta (Agent Environment)

Vnější prostředí, které agent prostřednictvím harnessu pozoruje a mění, například pracovní soubory, procesy, síťové služby a další systémové prostředky. ⁽²⁾

Průběžná integrace (CI)

Vývojová praxe, při níž se změny průběžně integrují a automaticky ověřují sestavením, testy a dalšími kontrolami. ⁽⁴²⁾

Pull Request

Návrh na sloučení změn z jedné větve do jiné, kolem kterého GitHub soustřeďuje revizi, diskusi a automatické kontroly. ⁽⁴⁰⁾

Předání řízení (Handoff)

Vzor koordinace, při kterém aktivní agent předá další řízení specializovanému agentovi. ⁽⁵²⁾

Přepis (Transcript)

Uspořádaný historický záznam událostí vzniklých během **Agentní sezení (Session)**, například zpráv, akcí a výsledků nástrojů. ⁽²⁾

RAG

Architektura, ve které systém před generováním vyhledá relevantní informace z externího zdroje a poskytne je modelu jako další kontext. ⁽⁴⁶⁾

Revize (Review)

Revize je samostatná kontrola změny nebo výstupu proti explicitním požadavkům a kvalitativním kritériím před jeho přijetím.

Slop

Neformální označení pro nekvalitní digitální obsah, zejména obsah vytvořený umělou inteligencí. ⁽⁴¹⁾

Spouštění kódu (Code Execution)

Nástrojová schopnost umožňující vykonat program nebo příkaz a vrátit jeho skutečný výstup modelu. ⁽²⁰⁾

Správa verzí (Version Control)

Systém pro zaznamenávání a porovnávání historie změn souborů v čase. ⁽³⁸⁾

Stav (State)

Persistovaná reprezentace aktuálně platných pracovních skutečností a řídicích údajů běhu.

⁽²⁾

Subagent

Specializovaná agentní instance, které jiný agent nebo orchestrátor deleguje vymezenou dílčí úlohu. ⁽⁵²⁾

Swarm

Swarm označuje dynamicky koordinované paralelní provádění úlohy více subagenty pod orchestrujícím agentem nebo systémem; nejde o univerzální formální standard.

Systémový prompt (System Prompt)

Systémová instrukční vrstva, která vymezuje roli, pravidla a výchozí způsob chování modelu nebo agenta. ⁽⁴³⁾

Teplota (Temperature)

Teplota je parametr vzorkování, který u rozhraní, jež jej podporují, ovlivňuje náhodnost výběru dalších tokenů. ⁽¹²⁾

Token

Diskrétní jednotka vstupní nebo výstupní sekvence reprezentovaná identifikátorem ve slovníku tokenizéru. ⁽⁸⁾

Transformer

Architektura neuronové sítě založená na mechanismu pozornosti, který modeluje vztahy mezi prvky sekvence. ⁽⁷⁾

Vektorová reprezentace (Embedding)

Vícerozměrná vektorová reprezentace diskrétních prvků, v níž geometrické vztahy mohou zachycovat užitečné vztahy mezi reprezentovanými objekty. ⁽⁹⁾

Velký jazykový model (LLM)

Velký neuronový jazykový model trénovaný na rozsáhlých textových datech pro predikci a generování posloupností tokenů. ⁽⁶⁾

Vibe Coding

Způsob tvorby softwaru, při kterém člověk iteruje pomocí pokynů v přirozeném jazyce bez průběžné kontroly vygenerovaného kódu. ⁽³²⁾

Vkládání kontextu (Context Injection)

V této práci označuje Context Injection cílené vložení relevantních informací do aktivního kontextu až v okamžiku, kdy jsou potřebné pro aktuální krok. ⁽³¹⁾

Vyvolávání nástrojů (Tool Calling)

Mechanismus, kterým model místo běžné textové odpovědi vybere konkrétní **Nástroje (Tools)** a vytvoří strukturované argumenty pro jeho vyvolání. ⁽¹⁾

Vývoj řízený specifikací (Spec-Driven Development)

Přístup k AI-asistovanému vývoji, ve kterém explicitní specifikace řídí plánování, implementaci a ověřování změny. ⁽³⁵⁾

Větev (Branch)

Oddělená linie vývoje v systému správy verzí, která ukazuje na vlastní posloupnost commitů.

⁽³⁸⁾

Člověk ve smyčce (HITL)

Uspořádání automatizovaného procesu, ve kterém člověk v určených bodech poskytuje zpětnou vazbu, schválení nebo rozhodnutí. ⁽⁵¹⁾

Seznam zdrojů

1. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/research/building-effective-agents>
2. ANTHROPIC. Scaling Managed Agents: Decoupling the brain from the hands. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/managed-agents>
3. MICROSOFT AI ECONOMY INSTITUTE. Global AI Diffusion Q1 2026: Trends and Insights. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.microsoft.com/en-us/research/wp-content/uploads/2026/05/Microsoft-AI-Diffusion-Report-2026-Q1.pdf>
4. HEVNER, Alan R., MARCH, Salvatore T., PARK, Jinsoo a RAM, Sudha. Design Science in Information Systems Research. *MIS Quarterly*. 2004. Vol. 28, no. 1.
5. PEFERS, Ken, TUUNANEN, Tuure, ROTHENBERGER, Marcus A. a CHATTERJEE, Samir. A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*. 2007. Vol. 24, no. 3, p. 45–77. DOI [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302).
6. BROWN, Tom B., MANN, Benjamin, RYDER, Nick, SUBBIAH, Melanie, KAPLAN, Jared, DHARIWAL, Prafulla, NEELAKANTAN, Arvind, SHYAM, Pranav, SASTRY, Girish, ASKELL, Amanda, HERBERT-VOSS, Ariel, KRUEGER, Gretchen, HENIGHAN, Tom, CHILD, Rewon, RAMESH, Aditya, ZIEGLER, Daniel M., WU, Jeffrey, WINTER, Clemens, HESSE, Christopher, CHEN, Mark, SIGLER, Eric, LITWIN, Mateusz, GRAY, Scott, CHES, Benjamin, CLARK, Jack, BERNER, Christopher, MCCANDLISH, Sam, RADFORD, Alec, SUTSKEVER, Ilya a AMODEI, Dario. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33. Available from: <https://arxiv.org/abs/2005.14165>
7. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
8. SENNRICH, Rico, HADDOW, Barry a BIRCH, Alexandra. Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Online. 2016. P. 1715–1725. Available from: <https://aclanthology.org/P16-1162/>
9. MIKOLOV, Tomas, CHEN, Kai, CORRADO, Greg a DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. Online. 2013. Available from: <https://arxiv.org/abs/1301.3781>

10. OPENAI. Models | OpenAI Agents SDK. Online. 2026. [Accessed 22 září 2026]. Available from: <https://openai.github.io/openai-agents-python/models/>
11. VLLM PROJECT. vLLM: A High-Throughput and Memory-Efficient Inference Engine for LLMs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://docs.vllm.ai/en/stable/api/vllm/>
12. OPENAI. Create a model response | OpenAI API Reference. Online. 2026. [Accessed 22 září 2026]. Available from: <https://developers.openai.com/api/reference/cli/resources/responses/methods/create>
13. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
14. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
15. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
16. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
17. OPENAI. Sessions | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/sessions/>
18. SCHICK, Timo, DWIVEDI-YU, Jane, DESSÌ, Roberto, RAILEANU, Roberta, LOMELI, Maria, ZETTLEMOYER, Luke, CANCEDDA, Nicola a SCIALOM, Thomas. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 68539–68551. Available from: <https://arxiv.org/abs/2302.04761>
19. OPENAI. Introducing Structured Outputs in the API. Online. 2024. [Accessed 20 září 2026]. Available from: <https://openai.com/index/introducing-structured-outputs-in-the-api/>
20. ANTHROPIC. Code execution tool. Online. 2026. [Accessed 20 září 2026]. Available from: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/code-execution-tool>

21. AGACHE, Alexandru, DEACONESCU, Razvan, IORDACHE, Mihai, LUPU, Alexandra, RADU, Vlad, BUGA, Cezar, BALAN, Catalin a FLORESCU, Andreea. Firecracker: Lightweight Virtualization for Serverless Applications. In : *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Online. 2020. p. 419–434. Available from: <https://www.usenix.org/conference/nsdi20/presentation/agache>
22. AGENT SKILLS. Agent Skills Specification. Online. 2026. [Accessed 20 září 2026]. Available from: <https://agentskills.io/specification>
23. ANTHROPIC. Create plugins | Claude Code Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/plugins>
24. ANTHROPIC. Hooks reference | Claude Code Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/hooks>
25. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
26. ANTHROPIC. Connect Claude Code to tools via MCP. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/mcp>
27. OPENAI. Customization overview. Online. 2026. [Accessed 22 září 2026]. Available from: <https://developers.openai.com/docs/customization/overview>
28. ANTHROPIC. How Claude remembers your project | Claude Code Docs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://code.claude.com/docs/en/memory>
29. ANTHROPIC. Settings files and precedence | Claude Code Docs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://code.claude.com/docs/en/settings>
30. ANTHROPIC. Extend Claude with skills | Claude Code Docs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://code.claude.com/docs/en/skills>
31. ANTHROPIC. Effective context engineering for AI agents. Online. 2025. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
32. KARPATY, Andrej. A new kind of coding I call vibe coding. Online. 2025. [Accessed 20 září 2026]. Available from: <https://twitter.com/karpathy/status/1886192184808149383>
33. WILLISON, Simon. A quote from Andrej Karpathy. Online. 2025. [Accessed 20 září 2026]. Available from: <https://simonwillison.net/2025/Feb/6/andrej-karpathy/>
34. KRAVETS, David. A semantic history: How the term 'vibe coding' went from a tweet to prod. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.coderabbit.ai/blog/a-semantic-history-how-the-term-vibe-coding-went-from-a-tweet-to-prod>
35. BÖCKELER, Birgitta. Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessel. Online. 2025. [Accessed 21 září 2026]. Available from: <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>

36. SOMMERVILLE, Ian. *Software Engineering*. 10. Pearson, 2016. ISBN 978-0-13-394303-0.
37. GITHUB. Pull request reviews | GitHub Docs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://docs.github.com/en/pull-requests/reference/pull-request-reviews>
38. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
39. GITHUB. Branches | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/reference/branches>
40. GITHUB. About pull requests | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
41. CAMBRIDGE UNIVERSITY PRESS. AI slop. Online. 2026. [Accessed 20 září 2026]. Available from: <https://dictionary.cambridge.org/dictionary/english/ai-slop>
42. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
43. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
44. OPENAI. Custom instructions with AGENTS.md. Online. 2026. [Accessed 22 září 2026]. Available from: <https://developers.openai.com/docs/agent-configuration/agents-md>
45. JIANG, Huiqiang, WU, Qianhui, LIN, Chin-Yew, YANG, Yuqing a QIU, Lili. LLMingua: Compressing Context for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*. Online. 2023. Available from: <https://arxiv.org/abs/2310.05736>
46. LEWIS, Patrick, PEREZ, Ethan, PIKTUS, Aleksandra, PETRONI, Fabio, KARPUKHIN, Vladimir, GOYAL, Naman, KÜTTLER, Heinrich, LEWIS, Mike, YIH, Wen-tau, ROCKTÄSCHEL, Tim, RIEDEL, Sebastian a KIELA, Douwe. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33, p. 9459–9474. Available from: <https://arxiv.org/abs/2005.11401>
47. OWASP GENAI SECURITY PROJECT. LLM01:2025 Prompt Injection. Online. 2025. [Accessed 21 září 2026]. Available from: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
48. OWASP FOUNDATION. LLM Prompt Injection Prevention Cheat Sheet. Online. 2026. [Accessed 21 září 2026]. Available from: https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
49. OPENAI. Understanding prompt injections. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.com/safety/prompt-injections/>
50. OPENAI. Guardrails | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/guardrails/>

51. MOSQUEIRA-REY, Eduardo, HERNÁNDEZ-PEREIRA, Elena, ALONSO-RÍOS, David, BOBES-BASCARÁN, José a FERNÁNDEZ-LEAL, Ángel. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*. Online. 2023. Vol. 56, no. 4, p. 3005–3054. Available from: <https://doi.org/10.1007/s10462-022-10246-w>
52. OPENAI. Agent orchestration | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: https://openai.github.io/openai-agents-python/multi_agent/
53. WU, Qingyun, BANSAL, Gagan, ZHANG, Jieyu, WU, Yiran, LI, Beibin, ZHU, Erkang, JIANG, Li, ZHANG, Xiaoyun, ZHANG, Chi, LIU, Jue, AWADALLAH, Ahmed Hassan, WHITE, Ryen W., BURGER, Doug a WANG, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*. Online. 2023. Available from: <https://arxiv.org/abs/2308.08155>
54. MOONSHOT AI. Kimi K2.5: Visual Agentic Intelligence. Online. 2026. [Accessed 22 září 2026]. Available from: <https://www.kimi.com/en/blog/kimi-k2-5>
55. MARIUS, Patrik. DarkFactory commit e9c10221. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/commit/e9c10221b40589512d262a0edb95f709b923150c>
56. GITHUB ACTIONS. DarkFactory CI run 35616745304. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/actions/runs/35616745304>
57. MARIUS, Patrik. Request #359: finish the df production engine and retire Python orchestration. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/issues/359>
58. MARIUS, Patrik. omnis commit a53660a1. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/omnis/commit/a53660a1c0c6619f94768e5d520405052fb03df6>
59. GITHUB ACTIONS. omnis pipeline run 34708160162. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/omnis/actions/runs/34708160162>
60. MARIUS, Patrik. ChessWithQuests commit 50a50797. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/ChessWithQuests/commit/50a50797f29c2a636d973981993191a65df3d131>
61. GITHUB ACTIONS. ChessWithQuests pipeline run 34708180783. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/ChessWithQuests/actions/runs/34708180783>
62. MARIUS, Patrik. template-OdbornaPrace repository. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/template-OdbornaPrace>

63. MARIUS, Patrik. OdbornaPrace-mono repository. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/OdbornaPrace-mono>
64. MARIUS, Patrik. DarkFactory-Paper commit 5bc04974. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/commit/5bc04974f9aed0f55295389154124a09059ff35e>
65. GITHUB ACTIONS. DarkFactory-Paper CI run 35617820423. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820423>
66. GITHUB ACTIONS. DarkFactory-Paper Deploy Documentation run 35617820271. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820271>
67. GITHUB ACTIONS. DarkFactory-Paper Release run 35617820286. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820286>
68. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
69. ANTHROPIC. Harness design for long-running application development. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/harness-design-long-running-apps>